

BUILD-DOMAINS.md

Version : V1

Statut : Référence officielle de construction des domaines métier

Type : Build Blueprint

1. Objectif

Ce document définit les règles officielles de construction de tous les domaines métier de Vitala.

Chaque domaine doit suivre exactement cette structure.

Aucune implémentation ne peut utiliser une organisation différente.

2. Principes fondamentaux

Chaque domaine doit respecter les principes suivants :

- Domain-Driven Design
- Single Responsibility
- High Cohesion
- Low Coupling
- API First
- Offline First
- Security by Design

Le domaine est l'unité principale de construction de Vitala.

3. Définition d'un domaine

Un domaine représente une capacité métier autonome.

Exemples :

- UDI
- Flash
- Mission
- Trust

Chaque domaine possède :

- son modèle métier ;
- ses règles métier ;

- ses APIs ;
 - ses données ;
 - ses événements ;
 - ses tests.
-

4. Structure officielle d'un domaine

Chaque domaine doit être organisé selon les couches suivantes :

Domaine

Contient :

- Entités
- Objets de valeur
- Agrégats
- Services métier
- Événements métier
- Interfaces des repositories

Cette couche ne dépend d'aucune technologie.

Application

Contient :

- Cas d'utilisation
- Orchestration
- Validation métier
- Coordination des opérations

Elle ne contient pas de logique d'infrastructure.

Infrastructure

Contient :

- Implémentation des repositories
 - Accès à Supabase
 - Stockage
 - Services externes
 - Synchronisation
 - Adaptateurs
-

Présentation

Contient :

- écrans
- composants UI
- gestion des états
- navigation
- formulaires

La logique métier y est interdite.

Tests

Chaque domaine possède :

- tests unitaires ;
 - tests d'intégration ;
 - jeux de données de test lorsque nécessaire.
-

5. Responsabilités d'un domaine

Un domaine est propriétaire de :

- ses règles métier ;
- ses tables ;
- ses APIs ;
- ses événements ;
- ses validations.

Il ne peut pas modifier directement les données d'un autre domaine.

6. Communication entre domaines

Les domaines communiquent uniquement par :

- APIs officielles ;
- événements métier ;
- projections en lecture.

Les accès directs aux repositories d'un autre domaine sont interdits.

7. Construction d'un nouveau domaine

Chaque nouveau domaine doit suivre les étapes suivantes :

1. Définir les objectifs métier.
 2. Définir les agrégats.
 3. Définir les entités.
 4. Définir les objets de valeur.
 5. Définir les événements métier.
 6. Concevoir les APIs.
 7. Concevoir les tables.
 8. Définir les règles de sécurité.
 9. Prévoir les tests.
 10. Documenter le domaine.
-

8. Gestion des données

Chaque domaine possède :

- ses tables ;
- ses index ;
- ses politiques RLS ;
- ses migrations.

Les tables partagées sont interdites, sauf justification documentée.

9. Gestion des APIs

Chaque domaine expose uniquement ses propres APIs.

Une API doit :

- respecter son contrat officiel ;
 - être versionnée ;
 - être documentée ;
 - être testée.
-

10. Gestion des événements

Chaque domaine peut produire des événements métier.

Les événements doivent :

- être clairement nommés ;
- être immuables ;
- être documentés ;
- être traçables.

Ils ne remplacent jamais les APIs.

11. Gestion Offline

Si un domaine est disponible hors connexion, il doit définir :

- les données synchronisées ;
 - les données locales ;
 - les brouillons (Drafts) ;
 - les opérations Outbox ;
 - les stratégies de résolution des conflits.
-

12. Sécurité

Chaque domaine définit :

- les permissions de lecture ;
- les permissions d'écriture ;
- les règles RLS ;
- les validations de sécurité.

Toutes les opérations sensibles doivent être journalisées.

13. Performances

Chaque domaine doit :

- limiter les requêtes inutiles ;
- utiliser des index adaptés ;
- supporter la pagination ;
- optimiser les lectures fréquentes.

Les optimisations ne doivent jamais compromettre la lisibilité.

14. Évolutivité

Un domaine doit pouvoir évoluer sans impacter les autres domaines.

L'ajout d'une fonctionnalité ne doit jamais nécessiter la modification de plusieurs domaines lorsque cela peut être évité.

15. Checklist de validation d'un domaine

Avant qu'un domaine soit considéré comme terminé, les éléments suivants doivent être validés :

- objectifs métier définis ;
 - agrégats validés ;
 - entités définies ;
 - objets de valeur définis ;
 - APIs documentées ;
 - base de données créée ;
 - sécurité validée ;
 - synchronisation documentée ;
 - tests réussis ;
 - documentation à jour.
-

16. Modèle de référence

Tous les domaines Vitala doivent suivre le modèle établi par :

- UDI
- Flash
- Mission
- Trust

Les futurs domaines (Marketplace, Learning, Finance, Health, etc.) devront respecter exactement la même structure.

17. Critères de conformité

Un domaine est conforme uniquement si :

- il respecte l'architecture officielle ;
- il est autonome ;
- il ne possède aucune dépendance circulaire ;
- ses APIs sont conformes ;
- ses données sont isolées ;
- ses tests sont validés.

18. Conclusion

BUILD-DOMAINS.md constitue la norme officielle de construction des domaines métier de Vitala.

Il garantit que tous les domaines présents et futurs seront construits selon un modèle unique, cohérent, évolutif et compatible avec le Domain Model officiel.